Creating a Voting System

Olivier

Introduction

The voting system is an end-to-end verifiable electronic voting platform designed to ensure transparency, security, and privacy in elections. This document provides a comprehensive explanation of how the system works, including its core components, encryption mechanisms, and verification processes. We will also use examples and pseudo-code to illustrate the workflow.

1 Core Components

1.1 Election Setup

Before voting begins, the election must be configured. This includes defining questions, answers, and cryptographic keys.

- `questions`: A list of questions with possible answers.
- `public_key`, `private_key`: Generated for encryption and decryption.
- `voting_starts_at`, `voting_ends_at`: Define the voting period.

```
election = Election(
    uuid="unique-election-id",
    questions=[
        {
            "short_name": "Favorite Fruit",
            "answers": ["Apple", "Banana", "Cherry"],
            "min": 1,
            "max": 1
        }
    ],
    public_key=generate_public_key(),
    private_key=generate_private_key(),
    voting_starts_at="2023-11-01 08:00:00",
    voting_ends_at="2023-11-01 18:00:00"
)
```

Listing 1: Election Configuration

1.2 Voter Registration

Each voter is assigned a unique identifier (`voter_uuid`) and a hash (`voter_hash`) for anonymity.

- `voter_uuid`: Unique identifier for the voter.
- `voter_hash`: Hash of the voter's ID for privacy.

```
voter = Voter(
    election_uuid=election.uuid,
    uuid="unique-voter-id",
    voter_id="john.doe@example.com",
    name="John Doe"
)
voter.voter_hash = hash(voter.voter_id)
```

Listing 2: Voter Registration

1.3 Vote Encryption

When a voter submits their vote, it is encrypted using the election's public key. Each answer is represented as an `EncryptedAnswer`.

- Each answer is encrypted as $g^r \cdot m$, where r is random and m is the plaintext.
- Zero-knowledge proofs ensure the correctness of the encryption.

```
vote = EncryptedVote.fromElectionAndAnswers(
    election=election,
    answers=[[0], [1]]  # Voter selects "Banana" for the first question
)
```

Listing 3: Vote Encryption

1.4 Tallying Votes

Votes are aggregated homomorphically, meaning they are summed without decryption. This ensures privacy while allowing the final tally to be decrypted.

- Homomorphic addition: $E(m_1) \cdot E(m_2) = E(m_1 + m_2)$.
- Decryption factors are combined to reveal the final tally.

```
tally = Tally(election=election)
for encrypted_vote in votes:
    tally.add_vote(encrypted_vote)

decryption_factors, decryption_proofs = tally.decryption_factors_and_proofs(
    private_key)
final_tally = tally.decrypt_from_factors(decryption_factors, public_key)
```

Listing 4: Tallying Votes

1.5 Verification

The system allows voters and auditors to verify the integrity of the election. Voters can check if their vote was included in the tally, and auditors can verify the correctness of the decryption process.

1. Verify the election hash matches the voter's record.
2. Check the zero-knowledge proofs for each encrypted vote.

3. Confirm the final tally matches the decryption factors.

```
# Voter verifies their vote
vote.verify(election)

# Auditor verifies decryption proofs
assert tally.verify_decryption_proofs(decryption_factors, decryption_proofs,
    public_key, challenge_generator)
```

Listing 5: Verification

2 Example Workflow

Let us consider an election with one question: "What is your favorite fruit?" with three options: Apple, Banana, and Cherry.

1. **Setup**: - The election generates a public/private key pair. - Questions and answers are defined.
2. **Voting**: - Voter 1 selects "Banana." - Their vote is encrypted and submitted as an `EncryptedVote`.
3. **Tallying**: - Votes are aggregated homomorphically. - Trustees provide decryption factors to reveal the final tally.
4. **Results**: - The tally shows: - Apple: 10 votes - Banana: 15 votes - Cherry: 5 votes - "Banana" is declared the winner.

3 Security Features

3.1 End-to-End Verifiability

- Voters can confirm their vote is included in the tally. - Auditors can verify the correctness of the tally without compromising privacy.

3.2 Homomorphic Encryption

- Votes are summed without decryption, preserving voter anonymity.

3.3 Zero-Knowledge Proofs

- Prove the validity of operations (e.g., encryption, decryption) without revealing sensitive information.

—

4 Vote Privacy and Encryption

It ensures vote privacy through the use of **homomorphic encryption**. Each vote is encrypted using the election's public key before being submitted. For example, when a voter selects "Banana" for a question, the choice is encrypted into an `EGCiphertext` object, which is stored in the `choices` field of the `EncryptedAnswer` class.

Mathematical Formula: The encryption process uses the ElGamal cryptosystem:

$$E(m) = (g^r \mod p, y^r \cdot m \mod p)$$

where: - g: Generator of the group. - r: Random number. - y: Public key ($y = g^x \mod p$). - m: Plaintext message (e.g., g^1 for "Banana").

Example: When a voter selects "Banana," their vote is encrypted as follows:

```
randomness = utils.random.mpz_lt(pk.q)
encrypted_choice = pk.encrypt_with_r(plaintexts[1], randomness)
```

Listing 6: Encryption Example

Even if the server is hacked, the attacker cannot decrypt the votes without the private key. The counting process works because homomorphic encryption allows encrypted votes to be summed without decryption.

Homomorphic Addition:

$$E(m_1) \cdot E(m_2) = E(m_1 + m_2)$$

For instance, in the `Tally` class, the `add_vote` method aggregates encrypted votes homomorphically:

```
self.tally[question_num][answer_num] = encrypted_vote.choices[answer_num] *
    self.tally[question_num][answer_num]
```

Listing 7: Homomorphic Tallying

Only after all votes are tallied does the system decrypt the final result using decryption factors provided by trustees.

—

5 Account Anonymity and Voter Assignment

The system maintains voter anonymity by separating voter identities from their votes. The `Voter` class includes a `voter_id_hash` property, which hashes the voter's unique identifier (e.g., email) to ensure privacy.

Example: When a voter registers, their `voter_hash` is stored instead of their actual identity:

```
@property
def voter_id_hash(self):
    return utils.hash_b64(self.voter_id)
```

Listing 8: Voter Hashing

In the `CastVote` class, the `voter_hash` is used to verify that the vote belongs to a registered voter without revealing their identity:

```
if self.vote.election_uuid != election.uuid:
    issues.append("the vote's election UUID does not match")
```

Listing 9: Vote Verification

This approach ensures that the system can validate voter eligibility while maintaining anonymity.

—

6 Vote Verification and Integrity

It allows voters to verify that their vote was counted correctly through cryptographic proofs. After casting a vote, the `CastVote` class generates a `vote_hash`, which is a hash of the encrypted vote. Voters can compare this hash with the one published on the bulletin board to confirm their vote was included.

Example:

```
@property
def hash(self):
    s = to_json(self.toJSONDict())
    return utils.hash_b64(s)
```

Listing 10: Vote Hash Generation

Additionally, the `EncryptedAnswer` class includes zero-knowledge proofs (`individual_proofs` and `overall_proof`) to verify the correctness of the encryption without revealing the vote itself.

Example: The `verify` method in `EncryptedAnswer` checks these proofs:

```
if not choice.verify_disjunctive_encryption_proof(possible_plaintexts,
    individual_proof):
    return False
```

Listing 11: Proof Verification

Auditors can also verify the integrity of the tally using the `verify_decryption_proofs` method in the `Tally` class:

```
if not proof.verify(public_key.g, answer_tally.alpha, public_key.y, int(
    dec_factor)):
    return False
```

Listing 12: Tally Verification

—

7 Preventing Fake Accounts and Vote Changes

To prevent fake accounts, It relies on voter registration and hashing mechanisms. Each voter is assigned a unique `voter_uuid` and `voter_hash`, which are verified during the voting process.

Example: The `issues` method in the `CastVote` class checks whether the `vote.election_uuid` matches the election for which the vote is being cast:

```
if self.vote.election_uuid != election.uuid:
    issues.append("Invalid election UUID")
```

Listing 13: Fake Account Prevention

To address the risk of adding virtual voters, the system could publish a hashed list of registered voters' hashes, allowing citizens to verify that their hash is included without revealing identities.

—

8 Multiple Choices and Cryptographic Counting

It supports multiple-choice questions by encrypting each choice individually. For example, if a question allows selecting up to three options, the system generates an encrypted answer for each choice and aggregates them homomorphically.

Example: The `generate_plaintexts` method creates possible plaintexts for each answer:

```
plaintexts = EncryptedAnswer.generate_plaintexts(pk, min=0, max=3)
```

Listing 14: Plaintext Generation

The system uses homomorphic encryption to sum encrypted votes. For example, in the `Tally` class, the `add_vote` method multiplies encrypted choices homomorphically:

```
self.tally[question_num][answer_num] = encrypted_vote.choices[answer_num] *
    self.tally[question_num][answer_num]
```

Listing 15: Homomorphic Aggregation

—

9 Abstention and Virtual Voters

To handle abstention, the system could include an explicit "abstention" option in the ballot design. For example, in the `Election` class, each question could have an additional "abstain" choice:

```
question['answers'].append("Abstain")
```

Listing 16: Abstention Handling

To prevent the addition of virtual voters, the system could publish a hashed list of registered voters and require auditors to verify the number of votes against the number of registered voters.

—

10 Conclusion

This voting system demonstrates how cryptographic techniques can enhance the security and transparency of electronic elections. By leveraging homomorphic encryption, zero-knowledge proofs, and decentralized trust,It ensures that votes remain private while enabling verifiable results. This system serves as a robust foundation for secure and trustworthy digital voting platforms.